

Numerical Integration: from concepts to Implementation

Daniel Chou¹

¹Georgia Institute of Technology, School of Civil and Environmental Engineering, Atlanta, GA 30332, United States of America

ABSTRACT

Learning notes of Gauss-Legendre Method, codes could be downloaded at [github](#).

1 Gauss-Legendre Method

1.1 Basic concept

The Gauss-Legendre Method is a numerical technique for calculating the integration of a function $f(x)$. It is particularly useful when the integration does not have an analytical solution, or the analytical solution is difficult to derive. The core concept is to estimate the integral in the range $[-1, 1]$ by a linear combination of Quadrature Weights w_i and Quadrature Nodes x_i , mathematically expressed as:

$$\int_{-1}^1 f(x) dx = G[f(x)] := \sum_{i=1}^N w_i f(x_i), \text{ s.t. } \sum_{i=1}^N w_i = 2 \text{ and } \forall x_i \in [-1, 1]$$

The challenge is to determine the w_i and x_i . These can be obtained by writing equations that express the fact that the Gauss-Legendre numerical integration techniques needs to be exact up to a certain polynomial order. For example, selecting $N = 2$ yields four unknowns (w_1, w_2, x_1, x_2), which can be solved using:

$$G[f(x) = x^0] = \int_{-1}^1 x^0 dx = 2 = w_1 x_1^0 + w_2 x_2^0$$

$$G[f(x) = x^1] = \int_{-1}^1 x^1 dx = 0 = w_1 x_1^1 + w_2 x_2^1$$

$$G[f(x) = x^2] = \int_{-1}^1 x^2 dx = \frac{2}{3} = w_1 x_1^2 + w_2 x_2^2$$

$$G[f(x) = x^3] = \int_{-1}^1 x^3 dx = 0 = w_1 x_1^3 + w_2 x_2^3$$

The solutions are $(w_1, w_2) = (1, 1)$ and $(x_1, x_2) = \left(-\frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}\right)$. Thus, by using $(w_1, w_2) = (1, 1)$ and $(x_1, x_2) = \left(-\frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}\right)$ along with a known formulation of $f(x)$, we can obtain the solution for $\int_{-1}^1 f(x) dx$. When N is large, directly calculating w_i and x_i for Gauss-Legendre Quadrature becomes impractical, as it requires solving high-degree polynomial equations. Therefore, Quadrature Weights and Nodes are commonly provided in tables, which can be found in various resources. For programming convenience, an alternative approach for evaluating quadrature nodes and weights is discussed in [section 2](#).

1.2 Range of integration

To evaluate an integration over the range $[a, b]$ for the function $g(u)$, i.e., $\int_a^b g(u) du$, we perform a variable transformation. For a general variable transformation, we express u as a function of x , which maps the range $[a, b]$ to $[-1, 1]$:

$$\int_a^b g(u) du = \int_{-1}^1 g(u(x)) \left| \frac{du}{dx} \right| dx.$$

The term $\left| \frac{du}{dx} \right|$ represents the Jacobian of the transformation, which adjusts the integration measure to account for the scaling between the u and x domains.

For a linear transformation from $[a, b]$ to $[-1, 1]$, we define

$$u(x) := \frac{b-a}{2}x + \frac{b+a}{2}, \quad \text{where } u = a \text{ for } x = -1 \text{ and } u = b \text{ for } x = 1.$$

Under this transformation, the Jacobian becomes a constant:

$$\frac{du}{dx} = \frac{b-a}{2}.$$

Therefore, we can rewrite the integration as

$$\begin{aligned} \int_a^b g(u) du &= \int_{-1}^1 g\left(\frac{b-a}{2}x + \frac{b+a}{2}\right) \frac{b-a}{2} dx \\ &= \frac{b-a}{2} \int_{-1}^1 g\left(\frac{b-a}{2}x + \frac{b+a}{2}\right) dx. \end{aligned}$$

The integration over x in the range $[-1, 1]$ can then be evaluated using Gaussian quadrature. Denoting this integral as $G[g(\frac{b-a}{2}x + \frac{b+a}{2})]$, we have

$$\begin{aligned} \int_a^b g(u) du &= \frac{b-a}{2} G\left[g\left(\frac{b-a}{2}x + \frac{b+a}{2}\right)\right] \\ &= \frac{b-a}{2} \sum_{i=1}^N w_i^x g\left(\frac{b-a}{2}x_i + \frac{b+a}{2}\right) \end{aligned} \quad (1)$$

$$= \sum_{i=1}^N w_i^u g(u_i), \quad (2)$$

where

$$w_i^u = \frac{b-a}{2} w_i^x, \quad u_i = \frac{b-a}{2}x_i + \frac{b+a}{2}. \quad (3)$$

Equation (2) along with (3) provides the general expression for the Gaussian-Legendre quadrature method on an arbitrary interval $[a, b]$. Here, w_i^x and x_i in (1) are the weights and nodes typically given for the interval $[-1, 1]$ with the chosen number of points N .

1.3 Double integration

Equation 1 can be extended to multiple integrations. For double integration with variable transformations $u_1 \rightarrow x_1$ and $u_2 \rightarrow x_2$, the combined weights are given by $W_{ij} = w_i^{x_1} w_j^{x_2}$, as follows:

$$\int_a^b \int_c^d g(u_1, u_2) du_1 du_2 = \frac{b-a}{2} \frac{d-c}{2} \sum_{i=1}^N \sum_{j=1}^M W_{ij} \left(g\left(\frac{b-a}{2}x_{1(i)} + \frac{b+a}{2}, \frac{d-c}{2}x_{2(j)} + \frac{d+c}{2}\right) \right) \quad (4)$$

$$= \sum_{j=1}^M W'_{ij} \left(g\left(x'_{1(i)}, x'_{2(j)}\right) \right) \quad (5)$$

where

$$W'_{ij} = \left(\frac{b-a}{2} w_i^{x_1}\right) \left(\frac{d-c}{2} w_j^{x_2}\right), \quad x'_{1(i)} = \frac{b-a}{2}x_{1(i)} + \frac{b+a}{2}, \quad x'_{2(j)} = \frac{d-c}{2}x_{2(j)} + \frac{d+c}{2}$$

2 Advanced topic: Numerical evaluation of Quadrature Weights and Nodes

Although the nodes x_i and weights w_i for the Gauss-Legendre quadrature are available in various tables, using these tables can take up a lot of memory in practical programming. This section offers a different method. By using Newton's method, a well-known technique for finding the roots of equations, we can calculate the Quadrature Nodes x_i , as well as the w_i . This approach is more memory-efficient for programming tasks that involve numerical integration.

2.1 Newton-Raphson Method

Consider the case where we have a function $P(x)$, and we want to find solutions x_i that satisfy $P(x) = 0$. Newton's method uses an iterative process. By starting with an initial guess $x_i^{(0)}$, it recursively updates the solution using the following equation:

$$x_i^{(t+1)} = x_i^{(t)} - \frac{P(x_i^{(t)})}{P'(x_i^{(t)})} \quad (6)$$

where $P'(x_i^{(t)}) = \frac{d}{dx}P(x_i^{(t)})$ represents the derivative of $P(x)$ with respect to x at $x_i^{(t)}$. The iterative process is terminated when the absolute difference between the current and previous estimates is smaller than a predetermined threshold η :

$$\Delta = |x_i^{(t+1)} - x_i^{(t)}| < \eta \quad (7)$$

If the condition is met, the process is terminated. The crucial question is "What is the function $P(x)$ for Quadrature nodes x_i ?" The answer lies in the Legendre polynomials.

2.2 The Legendre Polynomial

For a case with N nodes, the quadrature nodes x_i are the roots of the N -th degree Legendre polynomial:

$$P_N(x) = \frac{1}{2^N N!} \frac{d^N}{dx^N} (x^2 - 1)^N = 0.$$

The primary objective is to find x_i such that $P_N(x_i) = 0$. However, directly computing the N -th derivative of $(x^2 - 1)^N$ in a programming context is impractical due to the computational complexity involved. The process requires handling very large numbers and executing an extensive number of operations, which can be significantly resource-intensive and susceptible to numerical errors, especially for high values of N . Therefore, we may use the recursive relation of Legendre polynomials; each N -degree Legendre polynomial can be expressed through its preceding two polynomials. This relationship is encapsulated in Bonnet's recursion formula:

$$P_N(x) = \frac{(2N-1)xP_{N-1}(x) - (N-1)P_{N-2}(x)}{N} \quad (8)$$

Equation 8 describes Bonnet's recursion formula. The derivative of it, $P'_N(x)$, can be derived as:

$$P'_N(x) = \frac{N(P_{N-1}(x) - xP_N(x))}{1-x^2} = \frac{N(xP_N(x) - P_{N-1}(x))}{x^2-1} \quad (9)$$

The details of the derivation can be found in the Mathematics Stack Exchange discussion titled "First derivative of Legendre polynomial," available at a provided URL: [Math Stack Exchange](#).

By utilizing $P_N(x_i)$ from Equation 8 and $P'_N(x_i)$ from Equation 9, we can iteratively find the roots of the Legendre polynomial using the update rule from Equation 6 and the termination criterion from Equation 7. The quadrature weight w_i is then calculated using the quadrature node x_i and the derivative of the Legendre polynomial $P'_N(x_i)$:

$$w_i = \frac{2}{(1-x_i^2)(P'_N(x_i))^2} \quad (10)$$

For a detailed explanation of Equation 10, refer to the Wikipedia article on [Gauss-Legendre quadrature](#).

2.3 The initial guess of quadrature nodes

The quadrature nodes x_i are located within the range $[-1, 1]$ by definition. When N is large enough, the behavior of node indices v.s. x_i curve is similar to the pattern of the cosine function within the range $[0, \pi]$, as shown in Figure 1. Therefore, the initial guess of the i^{th} node x_i^0 can be approximated as:

$$x_i^0 = \cos\left(\frac{\pi(i-h_1)}{N+h_2}\right) \quad (11)$$

where $i \in \{1, 2, \dots, N\}$ and (h_1, h_2) are hyperparameters that assist in the convergence of the algorithm.

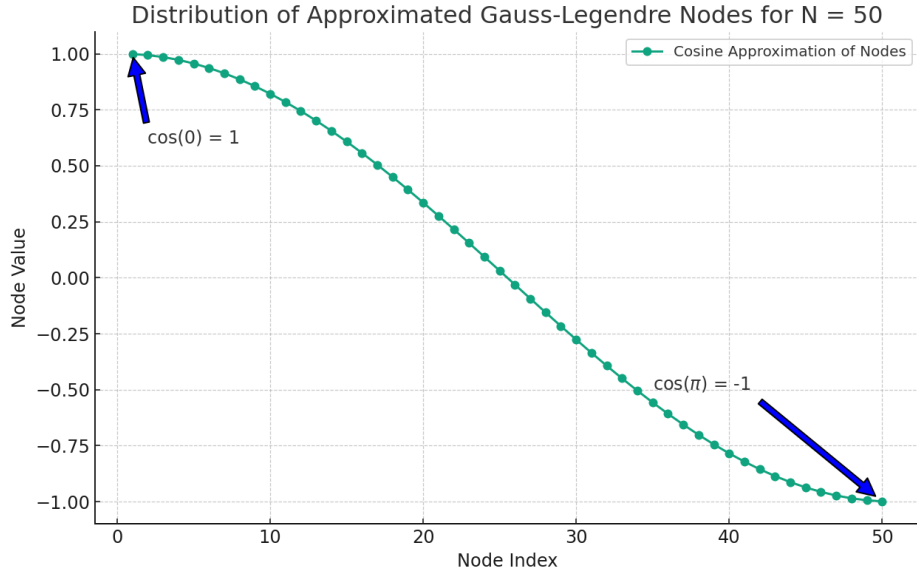


Figure 1. Example of $N = 50$, the behavior of quadrature node is similar to the cosine function.

2.4 Symmetry of Quadrature Nodes and Weights

As shown in Figure 1, the quadrature nodes x_i are symmetric about 0. For example, if N is even, such as $N = 2$, the nodes x_i are $\left\{-\frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}\right\}$, and the corresponding weights w_i are symmetric but not about 0, being $\{1, 1\}$. For an odd N , like $N = 3$, we have $x_i = \{-0.7746, 0, 0.7746\}$ and $w_i = \{0.5556, 0.8889, 0.5556\}$. This symmetry implies that only half of the nodes and weights need to be computed to obtain the full sequences of x_i and w_i . Suppose we create two arrays $\mathbf{x}$ and $\mathbf{w}$ to store the node and weight values, respectively. Here are the explicit steps for this computation:

1. With the number of nodes N , calculate the half-length as $M = \lfloor 0.5 \times (N + 1) \rfloor$. For example, for $N = 10$, $M = 5.5 \rightarrow M = 5$, and for $N = 9$, $M = 5$. For an odd N , the middle entry of $\mathbf{x}$ will be 0.
2. Iterate through $i = \{1, 2, \dots, M\}$. For the i^{th} entry of $\mathbf{x}$ and $\mathbf{w}$:
 - (a) Obtain x_i as the solution, assign $\mathbf{x}[i] = -x_i$, and then assign $\mathbf{x}[N + 1 - i] = x_i$. The initial guess based on the cosine function, as in Figure 1, ensures $x_i \geq 0$ within the half-length range of N . This also automatically ensures that the nodes in $\mathbf{x}$ are in ascending order.
 - (b) Calculate w_i via Equation 10, and assign $\mathbf{w}[i] = \mathbf{w}[N + 1 - i] = w_i$.

3 Summary of Algorithm

The steps to develop the algorithm based on the contents provided in this section are as follows:

1. Initialize two arrays $\mathbf{x}$ and $\mathbf{w}$ to store the nodes and weights, respectively.
2. Calculate the half-length M for the number of nodes N .
3. Iterate through $i = \{1, 2, \dots, M\}$:
 - (a) Make an initial guess for $x_i^{(t+1)}$ using Equation 11. Set the previous solution $x_i^{(t)} = x_i^{(t+1)} + 1$ to ensure a significant initial difference for the iterative process.
 - (b) Implement a while loop with the termination criterion as per Equation 7. Initialize $P_N(x) = 1$ and $P_{N-1}(x) = 0$. Within this loop, execute a for-loop over $j = \{1, 2, \dots, N\}$ to recursively estimate $P_N(x_i^{(t+1)})$ using Equation 8. In each iteration of this for-loop:

- i. Update $P_{j-2}(x)$ to the value of $P_{j-1}(x)$, and $P_{j-1}(x)$ to the value of $P_j(x)$.
 - ii. Apply Equation 8 to calculate the value of $P_N(x_i^{(t+1)})$.
- (c) Compute the derivative $P'_N(x_i^{(t+1)})$ using Equation 9. Update the node value: $x_i^{(t)} = x_i^{(t+1)}$, and then update $x_i^{(t+1)}$ using the Newton's method as $x_i^{(t+1)} = x_i^{(t)} - \frac{P_N(x_i^{(t)})}{P'_N(x_i^{(t)})}$ according to Equation 6.
- (d) Continue the iterative process until convergence is achieved to find the final value of x_i .
4. Calculate each weight w_i using Equation 10. Populate the arrays $\mathbf{x}$ and $\mathbf{w}$ with the computed x_i and w_i values, considering the symmetry of the nodes and weights.

4 Python Packages

A Python function `GLJointNW_general` is provided in the package `entropyLibDC`. The illustration is given as follows:

```
entropyLibDC.GLJointNW_general
```

General Gauss-Legendre method to generate nodes and weights for multiple integration, applicable in numerical integration tasks across multiple dimensions.

Parameters:

- **N** (list of int): Number of Gauss-Legendre quadrature points for each dimension.
- **Lower** (list of float): Lower bounds of integration for each dimension.
- **Upper** (list of float): Upper bounds of integration for each dimension.

Returns:

- **Gauss_points_weights** (numpy array): A multi-dimensional grid for numerical integration.
 - Shape: $(\text{prod}(\mathbf{N}), \text{len}(\mathbf{N})+1)$
 - Contains variables x_i within $[\text{Lower}[i], \text{Upper}[i]]$ for each i -th dimension, and the last column contains the joint weights $W_p \dots$ for integral calculations.
- **GLpw** (list of tuples): Contains nodes (Gauss points) and weights for each dimension, useful for separate or joint variable integration.
 - Each tuple: (Gauss points, weights) for individual dimension integration.

Example: To integrate a 2D function over $[0, 1] \times [0, 1]$ with 10 points in each dimension:

```
from entropyLibDC import *
N, Lower, Upper = [10, 10], [0, 0], [1, 1]
points_weights, individual_components = GLJointNW_general(N, Lower,
    ↪ Upper)
```